Pontificia Universidad Javeriana
Departamento de Ingeniería de Sistemas

Administración de Bases de Datos (DBA) – 2026-30

Proyecto final

Entrega 2: Límites PDET – carga en MongoDB e índices espaciales

Potencial solar en techos de los municipios PDET

Maria Camila Beltran

Jorge Fortich

Jose Eduardo Valeriano

Samuel Osorio Rodríguez

Profesor: Andrés Oswaldo Calderón Romero, Ph.D.

Noviembre 8 de 2026

Índice

1. Contexto **2**

2. Descarga del MGN y cruce con PDET **2**
2.1. Descarga del MGN 2025 . . . 2
2.2. Cruce con MunicipiosPDET.xlsx . . . 2
2.3. Verificación de los 170 municipios . . . 3

3. Validación, reparación y reproyección de geometrías **3**
3.1. Validación y reparación . . . 3
3.2. Reproyección y cálculo de área . . . 3
3.3. Exportación del GeoJSON . . . 4
3.4. Resultados . . . 5

4. Esquema, carga e índices en MongoDB **5**
4.1. Colecciones con validación `$jsonSchema` . . . 5
4.2. Carga idempotente con PyMongo . . . 6
4.3. Índices espaciales . . . 7
4.4. Evidencia del índice espacial . . . 7

5. Mapa de control y chequeo de calidad **7**
5.1. Mapa de control . . . 7
5.2. Chequeo de tamaño BSON . . . 8

6. Conclusiones y próximos pasos **9**

1. Contexto

Esta entrega corresponde a la **semana 2** del plan de implementación definido en la Entrega 1 (sección 3): cargar los límites de los 170 municipios PDET en MongoDB y dejar evidencia del índice espacial que soporta las consultas sobre ellos. El diseño del esquema, el diagrama E-R y el diccionario de datos no cambian respecto a la Entrega 1 y no se repiten aquí; se referencian cuando hace falta.

El flujo de esta semana tiene cuatro pasos encadenados por `src/municipios/run.py` (lo que corre `make municipios`): descargar el MGN 2025, cruzarlo con `MunicipiosPDET.xlsx`, validar y reparar las geometrías, y finalmente crear las colecciones, cargarlas e indexarlas en MongoDB.

2. Descarga del MGN y cruce con PDET

Esta sección cubre `descarga.py`, `cruce.py` y `verificacion.py`: traer el MGN 2025, quedarse solo con los municipios PDET y confirmar que los 170 cruzaron.

2.1. Descarga del MGN 2025

El geoportal del DANE genera la descarga desde una página web (no hay un enlace directo y estable al ZIP), así que `descargar_mgn()` admite dos formas de dar el archivo: uno ya puesto a mano en data/raw/mgn_2025/, o una URL directa opcional (`MGN_URL_DIRECTA` en `.env`). Para esta entrega se usó la primera opción, con el archivo descargado manualmente desde el Centro de Datos Geoestadísticos del DANE:

- **Fuente:** https://geoportal.dane.gov.co/servicios/descarga-y-metadatos/datos-geoestadisticos/?cod=111
- **Archivo:** `MGN2025_MPIO_GRAFICO.zip` (MGN2025, nivel municipio, formato Shapefile), renombrado localmente a `MGN2025.zip`.

El script calcula el SHA-256 del ZIP, lo descomprime en data/raw/mgn_2025/extraido/ y registra la fuente en la colección `fuentes` (`_id="mgn"`): URL de origen, versión, CRS leído del `.prj` de la capa, fecha de descarga y el checksum. Esta ejecución quedó registrada con:

Listing 1: Checksum verificado del ZIP descargado.

```
SHA-256: a3c01393059be8f2947de5e0a828faa4c7a43119819a667a8124854340bc141f
```

2.2. Cruce con MunicipiosPDET.xlsx

`cruzar_con_pdet()` lee data/raw/MunicipiosPDET.xlsx (el listado oficial de los 170 municipios PDET) y la capa municipal completa del MGN, rellena los códigos DANE a 5 dígitos con ceros a la izquierda (varios municipios de Antioquia vienen sin el cero inicial en el Excel) y cruza ambas tablas por `cod_mpio`. El resultado es un `GeoDataFrame` solo con los municipios PDET, que se guarda en data/processed/municipios_cruzados.gpkg para que `geometria.py` lo use en el siguiente paso (sección 3).

```
Proyecto-DBA infra/setup-entrega2 ❱ python -m src.municipios.cruce
Capa MGN: data/raw/mgn_2025/extraido/MGN_ADM_MPIO_GRAFICO.shp
Cruzaron 170 de 170 municipios PDET.
```

Figura 1: Ejecución de `cruce.py`: localiza la capa municipal del MGN y cruzan los 170 municipios PDET.

2.3. Verificación de los 170 municipios

`verificar_cobertura()` es un control independiente del cruce: vuelve a contar desde el Excel y la capa cruzada, para detectar si algún municipio PDET se quedó por fuera o si el Excel trae códigos repetidos.

Tabla 1: Tabla de control de `verificacion.py`.

Verificación	Esperado	Obtenido	OK
Municipios en el Excel (códigos únicos)	170	170	✓
Códigos repetidos en el Excel	0	0	✓
Municipios que cruzaron con el MGN	170	170	✓
Subregiones PDET en el resultado	16	16	✓
Departamentos en el resultado	19	19	✓

Los 170 municipios PDET cruzaron sin excepciones, en 16 subregiones y 19 departamentos, tal como se definió en el Excel oficial. El detalle de control por municipio queda en data/processed/verificacion_control.csv.

3. Validación, reparación y reproyección de geometrías

Esta sección cubre `geometria.py` y `exportar.py`. Su entrada es la capa que deja el cruce de (data/processed/municipios_cruzados.gpkg, sección 2) y su salida es data/processed/municipios_pdet.geojson, el archivo que se carga en la colección `municipios`. Como `run.py` llama a cada paso sin pasarle datos, los pasos se comunican por archivos en `data/processed/`, y cada uno puede volver a correrse por separado.

3.1. Validación y reparación

MongoDB rechaza un polígono inválido (por ejemplo, con bordes que se cruzan) en el momento de insertarlo en una colección con índice `2dsphere`, así que la validez se revisa antes de la carga. `validar_y_reparar()` recorre los 170 municipios y para cada uno:

- Revisa `is_valid` con Shapely y, si la geometría es inválida, guarda el motivo que reporta `is_valid_reason`.
- Repara las inválidas con `shapely.make_valid`. Como `make_valid` puede devolver una colección con líneas o puntos sueltos, se conservan solo las partes poligonales, para que el resultado sea siempre `Polygon` o `MultiPolygon`, como exige el diccionario de datos.
- Registra el resultado en el subdocumento `calidad`: `valida` (cómo venía de origen), `reparada` (si hubo que tocarla) y `n_vertices`.

Las 170 geometrías del MGN 2025 llegaron válidas, así que no fue necesario reparar ninguna. La reparación queda implementada y se aplica también después de reproyectar (sección 3.2), donde el redondeo de coordenadas podría introducir errores mínimos.

3.2. Reproyección y cálculo de área

Se usan dos sistemas de referencia, tal como se definió en la Entrega 1:

- **Área en EPSG:9377** (MAGNA-SIRGAS Origen Nacional). `area_km2` se calcula en esta proyección plana oficial, que está en metros. Calcular el área en grados no tiene sentido físico.
- **Almacenamiento en EPSG:4326** (WGS84). GeoJSON y los índices `2dsphere` exigen coordenadas `[longitud, latitud]` en WGS84. GeoPandas reproyecta con `always_xy=True`, así que el orden queda correcto.

Después de reproyectar, `reproyectar_y_calcular_area()` aplica tres ajustes y dos controles:

- Ajusta las coordenadas a una rejilla de 10^{-6} grados (unos 10 cm) con `shapely.set_precision`, que reduce el tamaño del archivo sin romper la validez, y elimina vértices repetidos.
- Orienta los anillos según el estándar GeoJSON (RFC 7946): exterior en sentido antihorario e interiores en sentido horario.
- Cuenta los vértices de cada municipio (`calidad.n_vertices`).
- **Control de orden lon/lat:** verifica que todas las coordenadas caigan dentro del rectángulo de Colombia. Si la longitud y la latitud quedaran invertidas, el script se detiene.
- **Control de validez final:** vuelve a revisar `is_valid` sobre las 170 geometrías ya reproyectadas.

Como verificación independiente, se comparó nuestra `area_km2` con el área oficial que trae el MGN en el campo `mpio_narea`. La diferencia máxima entre ambas fue de $2{,}2 \times 10^{-5}$ % y el área total coincide (389.182 km^2), lo que confirma que el cálculo en EPSG:9377 es correcto.

3.3. Exportación del GeoJSON

`exportar.py` escribe un `FeatureCollection` con un `Feature` por municipio. Cada uno lleva en `properties` los campos de la colección `municipios` (Entrega 1, sección 5.1), de modo que `carga.py` solo tiene que juntar `properties` y `geometry` en un documento. Antes de escribir, el script valida que no haya códigos DANE repetidos, que todos tengan 5 dígitos con el cero inicial y que solo haya geometrías `Polygon` o `MultiPolygon`. El identificador de la subregión (`subregion_pdet.id`, de 1 a 16) se asigna en orden alfabético, para que sea el mismo en cada ejecución. A diferencia de `data/raw/`, este archivo sí se versiona en el repositorio, para que el resto del equipo no dependa de volver a descargar el MGN.

Listing 2: Propiedades de un municipio en el GeoJSON exportado (se omite la geometría).

```
{
  "_id": "05031",
  "nombre": "AMALFI",
  "cod_dpto": "05",
  "departamento": "ANTIOQUIA",
  "subregion_pdet": { "id": 3, "nombre": "BAJO CAUCA Y NORDESTE ANTIOQUEÑO" },
  "area_km2": 1208.3462,
  "calidad": { "valida": true, "reparada": false, "n_vertices": 15106 },
  "fuente_id": "mgn"
}
```

3.4. Resultados

Tabla 2: Resultados de la validación, reproyección y exportación.

Indicador	Valor
Municipios procesados	170
Geometrías válidas de origen	170
Geometrías reparadas	0
Área total de los municipios PDET	389.182 km^2
Municipio más grande	Solano (Caquetá), 42.234 km^2
Municipio más pequeño	Chalán (Sucre), 84 km^2
Vértices en total	1.551.994
Vértices del municipio más complejo	51.568 (Solano)
Mediana de vértices por municipio	7.000
Documento más pesado (Solano)	1,74 MB (límite de MongoDB: 16 MB)
Tamaño de `municipios_pdet.geojson`	37,1 MB

El documento más pesado ocupa 1,74 MB (10,9 % del límite), muy por debajo del límite de 16 MB por documento de MongoDB, así que ningún municipio tuvo que simplificarse ni partirse. Este tamaño se confirma de forma independiente con `calidad.py` (sección 5.2) y con `scripts/chequeo_bson.py`. El detalle por municipio (validez, motivo, vértices y área) queda en `data/processed/reporte_geometrias.csv`.

```
(.venv) C:\Users\Usuario\Documents\Proyecto-DBA>python -m src.municipios.geometria
[geometria] 170 municipios revisados; 0 inválidos reparados.
[geometria] Resumen:
municipios               170
validas_de_origen        170
reparadas                  0
area_total_km2        389182.0
vertices_total         1551994
vertices_max             51568

(.venv) C:\Users\Usuario\Documents\Proyecto-DBA>python -m src.municipios.exportar
[exportar] 170 municipios -> C:\Users\Usuario\Documents\Proyecto-DBA\data\processed\municipios_pdet.geojson (37.1 MB)

(.venv) C:\Users\Usuario\Documents\Proyecto-DBA>
```

Figura 2: Ejecución de `geometria.py` y `exportar.py`: 170 municipios revisados, ninguno inválido, y el GeoJSON exportado.

4. Esquema, carga e índices en MongoDB

Esta sección cubre las cuatro partes: `schema.py`, `carga.py`, `indices.py` y `evidencia.py`. El código completo está en `src/municipios/`; aquí se explica qué hace cada uno y se muestra evidencia de que funciona contra una base MongoDB real.

4.1. Colecciones con validación `$jsonSchema`

`schema.py` crea `municipios` y `fuentes` con un validador `$jsonSchema` derivado directamente del diccionario de datos de la Entrega 1 (sección 5): campos requeridos, tipos BSON y, para `subregion_pdet` y `calidad`, la forma exacta de los subdocumentos embebidos. Si la colección ya existe (por ejemplo al volver a correr `make schema`), `create_collection` lanza

`CollectionInvalid` y el script lo captura sin fallar: no se toca el validador ni los datos que ya haya, para no arriesgarse a reescribir un esquema en uso sin coordinarlo antes con el equipo.

En `fuentes`, `periodo_imagenes` y `n_registros` aceptan `null` además de su tipo. `periodo_imagenes` no aplica al MGN (es un límite administrativo, no una imagen satelital con fecha de captura), y `n_registros` es un dato calculado: `descarga.py` todavía no sabe cuántos municipios van a cargarse cuando registra la fuente, así que `carga.py` lo completa después de insertar (sección 4.2). Con el validador original, estrictamente igual al diccionario de datos, la inserción de `descarga.py` fallaba con el error 121 apenas se intentaba registrar la fuente `mgn`.

```
Proyecto-DBA infra/setup-entrega2 ❱ python -m src.municipios.schema
Colección `municipios` creada con validación $jsonSchema.
Colección `fuentes` creada con validación $jsonSchema.
```

Figura 3: Ejecución de `schema.py`: crea ambas colecciones sin errores. Al volver a correrlo con las colecciones ya creadas, el script detecta que existen y no las modifica.

Como evidencia de que el validador realmente se aplica, se intentó insertar un documento incompleto directamente por `mongosh`: MongoDB lo rechaza con el código de error 121 y lista los campos que faltan.

```
Work  1:Proyecto-DBA  2:Proyecto-DBA  3:Proyecto-DBA  4:Tesis                omarchy

upme_solar> db.municipios.insertOne({_id: "99999", nombre: "x"})
Uncaught:
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: '99999',
  details: {
    operatorName: '$jsonSchema',
    schemaRulesNotSatisfied: [
      {
        operatorName: 'required',
        specifiedAs: {
          required: [
            '_id',
            'nombre',
            'cod_dpto',
            'departamento',
            'subregion_pdet',
            'area_km2',
            'geometry',
            'calidad',
            'fuente_id'
          ]
        },
        missingProperties: [
          'area_km2',
          'calidad',
          'cod_dpto',
          'departamento',
          'fuente_id',
          'geometry',
          'subregion_pdet'
        ]
      }
    ]
  }
}
upme_solar>
```

Figura 4: Un documento sin los campos obligatorios es rechazado por el validador `$jsonSchema`.

4.2. Carga idempotente con PyMongo

`carga.py` lee `data/processed/municipios_pdet.geojson` (la salida de `exportar.py`, sección 3.3) y arma un documento por `Feature`: las `properties` más `geometry`. En vez de `insert_many`, usa `bulk_write` con una operación `UpdateOne({_id: ...}, {$set: doc}, upsert=True)` por municipio, así que volver a correrlo no duplica documentos ni falla si alguno ya existe; simplemente lo deja igual o lo actualiza. Al terminar, actualiza `fuentes.mgn.n_registros` con el total cargado y registra la ejecución en `bitacora_cargas` (`etapa="carga"`, con `inicio`, `fin`, `registros_ok` y `registros_error`).

```
Proyecto-DBA infra/setup-entrega2 ❱ python -m src.municipios.carga
[carga] 170 municipios procesados; 170 en la colección.
```

Figura 5: Ejecución de `carga.py`: los 170 municipios quedan en la colección.

4.3. Índices espaciales

`indices.py` crea dos índices sobre `municipios`, los que pide el diccionario de datos (Entrega 1, sección 5.5): `2dsphere` sobre `geometry`, para las consultas espaciales (`$geoIntersects`, `$geoWithin`) que se usarán en las entregas 3 y 4 para asignar edificios a municipios, y un índice simple sobre `subregion_pdet.id` para los resúmenes por subregión.

```
Proyecto-DBA infra/setup-entrega2 ❱ python -m src.municipios.indices
[indices] creados: geometry_2dsphere, subregion_pdet.id_1
```

Figura 6: Ejecución de `indices.py`: `geometry_2dsphere` y `subregion_pdet.id_1` creados.

4.4. Evidencia del índice espacial

`evidencia.py` prueba que el índice `2dsphere` realmente se usa, no que simplemente exista. Toma un punto interior de un municipio del GeoJSON exportado (`representative_point()` de Shapely, que cae dentro del polígono aunque sea cóncavo) y corre una consulta `$geoIntersects` con ese punto. Dos verificaciones:

- `.explain()` sobre la consulta: las etapas del plan ganador deben incluir `IXSCAN` y no `COLLSCAN`, es decir, que Mongo usó el índice en vez de revisar las 170 geometrías una por una.
- El resultado de la consulta debe ser exactamente un municipio, y debe ser el mismo del que se tomó el punto.

```
Proyecto-DBA infra/setup-entrega2 ❱ python -m src.municipios.evidencia
[evidencia] etapas del plan: ['FETCH', 'IXSCAN'] (se espera IXSCAN, no COLLSCAN)
[evidencia] punto de control: AMALFI (05031) en [-74.95147, 7.02048]
[evidencia] $geoIntersects devolvió: [{'_id': '05031', 'nombre': 'AMALFI'}]
[evidencia] OK: el índice 2dsphere se usa (IXSCAN) y el geoIntersects devuelve el municipio correcto.
```

Figura 7: `evidencia.py`: el plan usa `IXSCAN` y `$geoIntersects` devuelve el municipio correcto para el punto de control.

5. Mapa de control y chequeo de calidad

Esta sección cubre `mapa.py` (t04) y `calidad.py` (t07): una evidencia visual de que los 170 municipios y sus 16 subregiones quedaron correctos, y la confirmación de que ningún documento se acerca al límite de 16 MB de MongoDB antes de cargarlos.

5.1. Mapa de control

`generar_mapa_control()` lee `data/processed/municipios_pdet.geojson` (la salida de `exportar.py`, sección 3.3) y dibuja los 170 municipios con GeoPandas y Matplotlib, coloreados por subregión PDET. Sirve como control visual rápido: si faltara un municipio o alguno quedara

con una geometría rota, se notaría de inmediato en el mapa. El archivo se guarda en `docs/mapa_control_municipios.png`.

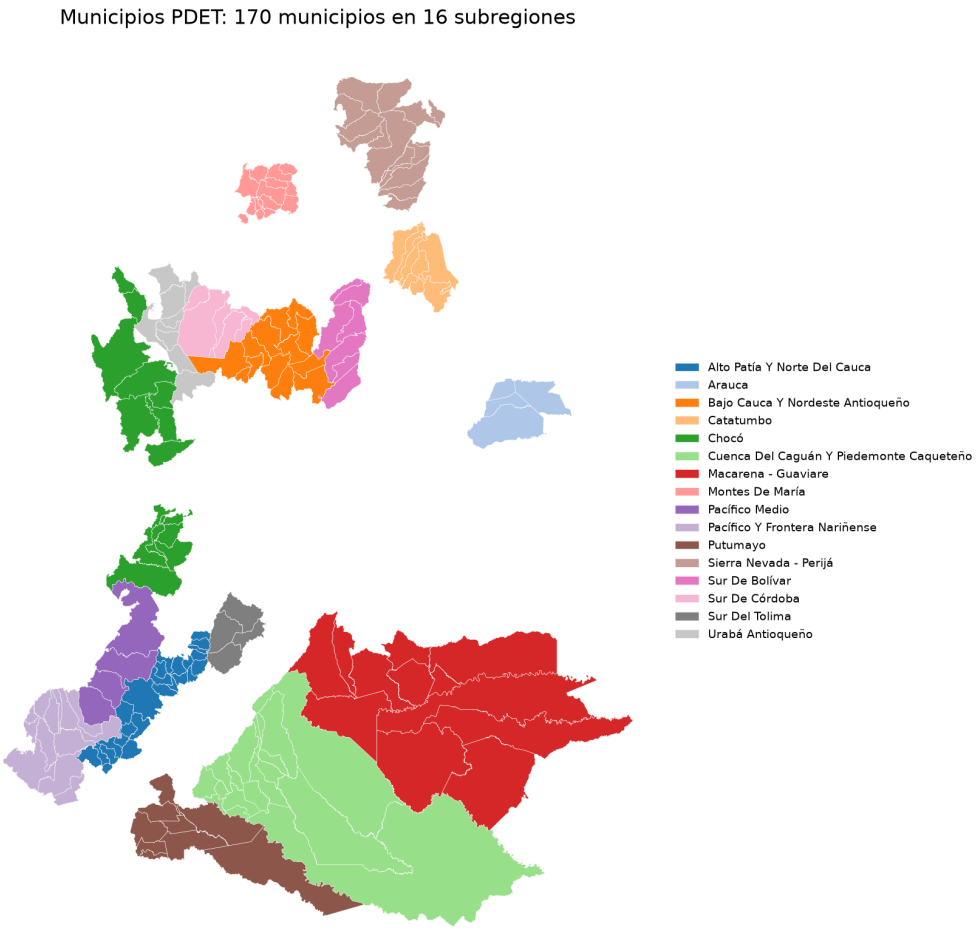


Figura 8: Los 170 municipios PDET coloreados por sus 16 subregiones.

El mapa confirma lo que ya mostró la tabla de control de `verificacion.py` (sección 2.3): los 170 municipios están completos y agrupados en las 16 subregiones esperadas, sin huecos ni geometrías sueltas fuera de su región.

5.2. Chequeo de tamaño BSON

MongoDB rechaza cualquier documento que supere 16 MB. `chequear_tamanos()` mide, para cada municipio, el tamaño BSON exacto del documento tal como lo guarda `carga.py` (`properties` más `geometry`), usando `bson.encode()` directamente en vez de estimar por el tamaño del GeoJSON en disco. Reporta el municipio más pesado y marca con una alerta cualquiera que pase el 90 % del límite (`config.yaml -> municipios.max_bson_bytes`); si alguno superara el 100 %, el script falla para que no se intente cargar.

Listing 3: Salida de `calidad.py` sobre los 170 municipios PDET.

```
[calidad] 170 documentos; el mayor es 18756 con 1742528 bytes (10.9% del límite de 16000000).
```

El municipio más pesado es Solano (Caquetá, código DANE 18756) con 1.742.528 bytes (10,9 % del límite), coherente con ser también el municipio con más vértices (sección 3.4). Ningún municipio se acercó al umbral de alerta del 90 %, así que no hizo falta simplificar ninguna geometría para esta entrega.

6. Conclusiones y próximos pasos

El objetivo de esta entrega era cargar los 170 municipios PDET en MongoDB con evidencia de que las consultas espaciales usan el índice, y se cumplió de punta a punta: los 170 municipios del MGN 2025 cruzaron con el listado oficial sin excepciones (16 subregiones, 19 departamentos), sus geometrías llegaron válidas (ninguna necesitó repararse), y el GeoJSON exportado (37,1 MB, documento más pesado 1,74 MB) se cargó completo en la colección `municipios` con los índices `2dsphere` y `subregion_pdet.id`. La evidencia de `evidencia.py` (sección 4.4) confirma que una consulta `$geoIntersects` resuelve con `IXSCAN`, no con un recorrido completo de la colección, y el mapa de control (sección 5.1) deja ver los 170 municipios agrupados correctamente en sus subregiones.

El flujo completo quedó encadenado en `src/municipios/run.py`, de modo que `make municipios` reproduce todo el proceso –descarga, cruce, verificación, geometría, esquema, carga, índices y evidencia– desde cero. Los únicos datos que se versionan son `data/raw/MunicipiosPDET.xlsx` (insumo pequeño y estable) y `data/processed/municipios_pdet.geojson` (la salida final, 37 MB), para que ningún integrante del equipo dependa de volver a descargar el MGN completo solo para probar su parte.

Como aprendizajes del proceso: el validador `$jsonSchema` derivado del diccionario de datos solo tiene sentido si los campos *calculados en otra etapa* (como `fuentes.n_registros`) se declaran opcionales o nulos, porque exigirlos desde el primer insert bloquea el flujo por el que se van completando los datos (sección 4.1); y separar cada tarea en su propio archivo de `src/municipios/`, comunicándose por archivos intermedios en `data/processed/`, permitió que las cuatro personas trabajaran en paralelo sin bloquearse, a costa de tener que coordinar manualmente el orden de `run.py` cuando una etapa pasó a depender de la salida de otra (como ocurrió con `calidad.py`, que necesita el GeoJSON que deja `exportar.py`).

Para la semana 3 (huellas de edificios), el plan de implementación de la Entrega 1 define el siguiente paso: descargar y prefiltrar Google Open Buildings y Microsoft Building Footprints recortados al área de los 170 municipios PDET (usando el `geometry` y el índice `2dsphere` que ya quedaron listos en esta entrega), cargar `edificios_google` y `edificios_microsoft`, y dejar los índices `2dsphere` sobre `centroide` y `geometry` que se usarán para asignar cada edificio a su municipio.